Streaming Communication for Scalable Data Exchange in DYNAMOS

Ruben Stoop
Complex Cyber-Infrastructure (CCI)
University of Amsterdam
Amsterdam, the Netherlands

***Abstract*—This thesis investigates how streaming communication can be introduced into DYNAMOS, a policy-driven middleware for data exchange based on dynamically composed microservices. The current system relies on unary gRPC request-response interactions, which become increasingly inefficient for large or continuous transfers. To ground the broader architectural and comparative study, the thesis first reviews candidate streaming mechanisms and establishes a controlled evaluation baseline in a common *Producer* → *Transformer* → *Sink* pipeline. The current empirical results compare client-streaming and unary gRPC across transfer efficiency, latency stability, overload behaviour, recovery characteristics, and resource cost. They show that client-streaming gRPC provides substantially higher sustained throughput, steadier pacing, and much faster restart recovery for long-running transfers, while unary gRPC maintains lower per-message latency and duplicate-free delivery in the current prototype. These findings define the trade-off space that subsequent broker comparisons and DYNAMOS integration work must address.**

***Index Terms*—streaming communication, microservices, gRPC, event streaming, policy-driven middleware, DYNAMOS**

I. Introduction

Many contemporary distributed systems operate in environments where data and computational resources are owned by different organizations. In such settings, data exchange must adhere to legal agreements, privacy regulations, and organizational policies. These constraints are common in domains such as healthcare, scientific research, and public-sector infrastructures, where unrestricted data exchange is not acceptable.

DYNAMOS (Dynamically Adaptive Microservice-based Operating System) is a data exchange middleware developed by the Complex Cyber-Infrastructure (CCI) group at the University of Amsterdam. The system implements a set of recurring data sharing archetypes that describe how data and computation are distributed among participants under policy constraints. DYNAMOS realizes these archetypes by dynamically composing chains of microservices that execute data sharing requests in compliance with formally evaluated policies. The middleware is designed to be adaptive, allowing architectural and execution decisions to change at runtime in response to policy requirements.

The current implementation of DYNAMOS uses gRPC for inter-service communication and relies exclusively on atomic request–response interactions. This communication model is suitable for small or bounded data transfers but becomes limiting when applied to large-scale or continuous data exchange. In practice, data in distributed systems is often produced and consumed incrementally, for example in sensor data streams, distributed analytics pipelines, or staged processing of large research datasets.

When request–response communication is used in such scenarios, systems must either buffer large volumes of data before transmission or split transfers into multiple independent requests. Buffering increases memory usage and can lead to idle time between producers and consumers. Repeated request–response interactions introduce additional coordination overhead. Both effects negatively impact scalability and resource utilization as data volumes and the number of participating services increase.

Streaming-based communication has been proposed as a solution to these limitations. Streaming allows data to be transferred and processed incrementally, enabling overlap between computation and communication and reducing peak memory usage. gRPC supports several streaming interaction patterns, including client-side, server-side, and bidirectional streaming.

Integrating streaming communication into DYNAMOS is not straightforward. Streaming relies on long-lived connections and continuous data flows, which must be reconciled with DYNAMOS' policy-driven execution model, dynamic microservice composition, orchestration decisions, and fault handling mechanisms. Without careful design, the introduction of streaming may interfere with policy enforcement or reduce the ability to evaluate and reason about system behavior.

The goal of this thesis is to investigate how streaming communication can be incorporated into DYNAMOS in a way that improves scalability and performance while preserving policy compliance and adaptive behavior. This involves analyzing existing streaming communication mechanisms, designing architectural extensions to the middleware, implementing streaming support, and evaluating the resulting system under realistic workloads.

The main research question addressed in this thesis is how streaming communication can be integrated into a dynamically adaptive, policy-driven microservice-based data exchange middleware such as DYNAMOS while preserving compliance guarantees and improving scalability and performance. To answer this overarching question, the thesis is structured around three sub-questions: which streaming communication

technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS; how streaming communication can be integrated into a microservice-based architecture while preserving dynamic service composition, policy-driven orchestration, and isolation guarantees; and what impact streaming communication has on scalability, performance, and robustness compared to a traditional request–response approach under realistic workloads in a microservice-based architecture.

The present document concentrates on the first empirical step in that broader programme. It narrows the design space through a literature-guided comparison, defines a reproducible benchmark for DYNAMOS-relevant streaming workloads, and reports the current gRPC baseline results. Later thesis stages build on this baseline by extending the comparison to broker-based transports and by evaluating architectural fit inside DYNAMOS itself.

II. Related Work

This section reviews existing literature related to streaming communication in microservice-based systems, with a particular focus on topics that are central to this thesis: streaming communication technologies and interaction models, architectural integration in adaptive microservice middleware, policy enforcement under continuous data exchange, resilience mechanisms for streaming workflows, and validation practices with respect to scalability and performance.

A. Streaming communication technologies and interaction models

DYNAMOS is implemented as a dynamically composed microservice system deployed on Kubernetes, using gRPC for inter-service communication and RabbitMQ for asynchronous messaging [1], [2]. This constrains the design space for introducing streaming functionality to a limited set of technologies that are compatible with containerized microservices and cloud-native deployment models. The primary question addressed in this body of work is which streaming communication mechanisms are suitable for large-scale or continuous data exchange, and under which conditions direct streaming RPC should be preferred over broker-based approaches.

gRPC provides native support for streaming through client-streaming, server-streaming, and bidirectional streaming RPCs. These interaction patterns differ in message flow control, lifetime of connections, buffering behaviour, and error propagation semantics. In the context of DYNAMOS, the choice of streaming pattern is not merely an API concern but has architectural consequences, as it affects memory usage, cancellation behaviour, and the ability to overlap communication and computation. The original DYNAMOS thesis identifies client-streaming as a promising mechanism for transferring large datasets incrementally, reducing peak memory pressure at the cost of increased handling complexity [1].

However, the use of long-lived streaming connections introduces operational challenges that are largely absent in unary request–response interactions. Starck and Ghofrani show that gRPC streaming complicates load balancing in Kubernetes environments, as persistent connections can cause traffic to become unevenly distributed across replicas [3]. This observation is particularly relevant for systems such as DYNAMOS that rely on horizontal scaling and ephemeral execution units. Complementary benchmarking work demonstrates that the performance characteristics of streaming gRPC are sensitive to runtime configuration and virtualization overhead, with measurable effects on latency and resource utilisation in containerized environments [4].

An alternative to direct service-to-service streaming is to externalise the data plane to a messaging or event streaming platform. Broker-based communication decouples producers and consumers in time, delegates buffering and persistence to the broker, and can simplify fan-out scenarios. Comparative benchmarks of Kafka, RabbitMQ, NATS, and ActiveMQ Artemis show that different brokers dominate under different workload characteristics, reinforcing that broker choice is highly workload dependent [5]. A cloud-native literature review comparing Kafka, Pulsar, and RabbitMQ further highlights trade-offs between throughput, reliability, operational complexity, and Kubernetes integration [6]. While these studies are not conducted in the context of policy-driven middleware, they provide structured criteria that are directly applicable when evaluating broker-based alternatives for DYNAMOS.

Several studies also emphasize that streaming design is influenced not only by technology choice but by the underlying communication model. DataXc contrasts push-based streaming approaches with pull-based designs and reports improved latency stability and throughput under consumer-controlled flow [7]. Although the application domain differs, the distinction between producer-driven and consumer-driven streaming is directly relevant to backpressure management and resource utilisation in DYNAMOS-compatible workflows.

Overall, existing literature provides a broad comparison of streaming technologies and interaction patterns, but these comparisons are typically conducted in generic microservice or stream-processing settings. There is limited work that evaluates these mechanisms under the specific constraints of dynamically composed, policy-driven middleware with ephemeral execution units, leaving open questions regarding their suitability in such environments.

B. Architectural integration of streaming in adaptive microservice middleware

Beyond the selection of a streaming mechanism, integrating streaming communication into an adaptive microservice architecture raises architectural questions related to modularity, orchestration, and lifecycle management. DYNAMOS is explicitly designed around dynamic microservice composition, where execution chains are generated and deployed per request based on policy evaluation and system state [1], [2]. These chains are short-lived by design, which improves isolation

and compliance guarantees but complicates the management of long-lived communication channels.

Communication in DYNAMOS is abstracted through a sidecar-based architecture, decoupling communication concerns from core microservice logic. The sidecar pattern is widely used in cloud-native systems to address cross-cutting concerns such as observability, security, and traffic management. Figueira and Coutinho demonstrate how sidecars can be integrated into self-adaptive microservice architectures on Kubernetes, enabling runtime adaptation without violating service modularity [8]. While their work does not explicitly address streaming data transfer, it illustrates how communication behaviour can be adapted dynamically at runtime.

Several middleware frameworks operationalise sidecars as first-class runtime components. Dapr provides a protocol-agnostic communication abstraction that supports service invocation, pub/sub, and state management through a sidecar runtime [8]. Dapr demonstrates that heterogeneous communication paradigms, including streaming and messaging, can be unified behind a stable interface. However, Dapr assumes relatively static service topologies and does not address dynamic chain composition driven by policy evaluation, limiting its applicability as a direct solution for DYNAMOS.

Existing architectural work therefore supports the feasibility of integrating streaming into sidecar-based microservice systems but does not address the combination of long-lived streaming connections with ephemeral, dynamically composed execution chains. This limitation is particularly relevant in systems that combine adaptivity with strict policy enforcement requirements.

C. Policy enforcement and compliance under continuous data exchange

Policy enforcement is a defining characteristic of DYNAMOS, where access rights, execution locations, and data-flow constraints are evaluated prior to execution and enforced through dynamically composed microservice chains [1], [2]. In the current architecture, policy enforcement is implicitly scoped to bounded request–response interactions. Introducing streaming communication challenges this assumption, as data transfer may span longer periods during which policies, system state, or execution context may change.

Neumaier et al. propose a policy-aware architecture for decentralized dataset exchange that emphasizes continuous monitoring and runtime validation against machine-readable policies [9]. While their work does not explicitly target streaming RPC, it introduces an important distinction between one-time policy clearance and ongoing compliance monitoring, which becomes critical when data is exchanged incrementally over long-lived connections.

At the communication layer, several studies propose enforcing security and access control through proxies or sidecars. Thiyagarajan et al. present a proxy-based approach for securing gRPC communication using mTLS, JWT authentication, and RBAC [10]. Although the focus is on security rather than data usage policy, the architectural approach is relevant, as it suggests that enforcement can be applied at communication boundaries and potentially at message granularity.

Despite these contributions, there is limited guidance on how policy enforcement should interact with long-lived streaming connections in systems that dynamically compose execution chains. In particular, existing work does not address how policy violations should be detected or handled mid-stream, or how streams should be terminated or adapted when compliance conditions change. These limitations highlight open challenges when introducing streaming into policy-driven middleware such as DYNAMOS.

D. Resilience, backpressure, and fault handling in streaming workflows

Streaming-based communication introduces operational challenges that are less pronounced in request–response systems. Long-lived data flows must tolerate partial failures, adapt to fluctuating load, and prevent uncontrolled resource growth through effective backpressure mechanisms. These challenges are compounded in DYNAMOS by dynamic microservice composition and ephemeral execution units.

Reactive microservice architectures emphasize asynchronous communication, non-blocking execution, and fault isolation as mechanisms for improving resilience under variable load [11]. Although this work does not focus on streaming RPC, its principles are directly applicable to streaming workflows, where failures and backpressure must be managed continuously rather than per request.

Studies on high-volume data processing in microservices highlight the importance of explicit flow control and decoupling between producers and consumers. Guntupalli and Vamshi show that event-driven architectures combined with messaging systems can mitigate bottlenecks and stabilize performance under sustained data rates [12]. While these systems typically rely on brokers rather than direct streaming RPC, they reinforce the need for explicit backpressure mechanisms in streaming data paths.

From a systems perspective, Gan and Delimitrou demonstrate that communication overhead and tail latency dominate the performance of large-scale microservice deployments [13]. Their findings underline that resilience and scalability issues often emerge from inter-service communication rather than computation, an effect that is amplified in streaming scenarios.

Existing literature provides well-established principles for resilience and fault tolerance in microservice systems, but these are typically studied either in request-oriented architectures or broker-based streaming platforms. There is limited work examining how these mechanisms interact with direct streaming RPC in dynamically composed microservice chains, leaving questions about how failures and backpressure should be handled in dynamically composed microservice systems.

E. Validation practices, scalability, and performance evaluation

Empirical evaluation of streaming communication often focuses on comparing streaming and non-streaming interaction

styles in terms of throughput, latency, and resource usage. Several benchmarking studies show that streaming RPCs outperform unary calls for large payloads and continuous data flows by enabling incremental processing and reducing buffering overhead [4]. Similar conclusions are drawn in broader comparisons between REST and gRPC, where binary protocols combined with streaming semantics demonstrate superior performance under load [14], [15].

Scalability in containerized streaming systems has been studied through systematic mapping and empirical evaluations. A mapping study on performance analysis techniques for streaming workloads in containerized microservices identifies network contention, scheduling overhead, and coordination costs as primary scalability bottlenecks [16]. Empirical studies of streaming pipelines deployed on Kubernetes further show that streaming approaches scale more gracefully than request-response models when concurrency increases, provided that backpressure and flow control are properly configured [3].

However, most existing evaluations assume static microservice topologies and fixed execution paths. Prior performance studies rarely consider adaptive middleware that dynamically constructs execution chains based on policy evaluation. As a result, there is limited empirical insight into how streaming communication scales when embedded within dynamically adaptive, policy-driven systems such as DYNAMOS. This lack of combined evaluation of streaming, scalability, and adaptivity motivates the experimental focus of this thesis.

III. Candidate Mechanisms and Benchmark Design

This section addresses *RQ1*: which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS? It combines a literature-backed comparison of candidate mechanisms with a controlled benchmark design that establishes a gRPC reference baseline before broker-based alternatives are added.

A. gRPC streaming

According to the official gRPC documentation, gRPC is based on service definitions in Protocol Buffers and supports four RPC styles: unary, server-streaming, client-streaming, and bidirectional streaming. For streaming calls, message ordering is preserved within each individual RPC, while deadlines, cancellation, metadata, and both synchronous and asynchronous APIs are supported by the runtime [17]. These properties make gRPC streaming the most direct extension of the current DYNAMOS communication model.

For microservice chains with predominantly point-to-point data transfer, client-streaming is the most natural first candidate because it lets a sender incrementally push chunks or records to a single downstream consumer. Bidirectional streaming becomes relevant when the receiver should provide progress feedback, pacing information, or explicit acknowledgements during transfer. The main advantages of gRPC streaming are low protocol overhead, direct reuse of protobuf contracts, and the absence of an additional broker layer. The disadvantages are that sender and receiver remain temporally coupled, durable replay is not provided by default, and backlog handling or crash recovery must be implemented at application level. Even so, prior work on asynchronous gRPC streaming demonstrates that carefully designed protocols can support efficient and even exactly-once style delivery patterns, which makes gRPC a strong experimental baseline rather than merely the status-quo option [4], [14], [18].

B. Apache Kafka

Apache Kafka is officially described as an open-source distributed event-streaming platform for high-performance pipelines, streaming analytics, data integration, and mission-critical applications. Its published core capabilities emphasise high throughput, durable storage, high availability, built-in stream processing, and large-scale horizontal scalability [19]. In architectural terms, Kafka exposes an append-only log abstraction in which messages are written to topics and partitions, allowing replay and independent consumption by multiple downstream readers [20].

Kafka is therefore the strongest candidate when temporal decoupling, replayability, or large consumer backlogs matter more than direct point-to-point simplicity. Its advantages are durable retention, repeated reads by independent consumers, and a mature ecosystem for analytics and downstream integration. The disadvantages are the extra operational footprint of a broker cluster, partition-local rather than global ordering, and a non-trivial tuning burden around acknowledgements, partitioning, batching, and replication. Recent review work also notes that Kafka benchmarks are frequently hard to reproduce because configuration details are underreported, which strengthens the case for a carefully controlled thesis benchmark [20]–[22].

C. RabbitMQ

Earlier comparative work positions RabbitMQ as a routing-oriented messaging system that offers flexible exchanges, acknowledgements, and strong support for enterprise-style asynchronous messaging [20], [23]. Current RabbitMQ documentation additionally introduces *streams* as an append-only, non-destructive, always persistent and replicated data structure that complements traditional queues rather than replacing them. Stream consumers can attach at arbitrary offsets or timestamps, replay data, and scale out via super streams when partitioning is needed [24].

This makes RabbitMQ attractive as a hybrid comparison point between classic broker semantics and newer log-like stream semantics within the same ecosystem. Its advantages are flexible routing, explicit acknowledgements, and a gradual migration path from queue-oriented messaging to append-only streams. The limitations are that classic RabbitMQ semantics are not optimised for replay-heavy logs, while stream mode introduces its own replication and partitioning trade-offs. RabbitMQ should therefore be evaluated not only on raw throughput, but also on how well it balances routing flexibility,

operational continuity, and stream-specific behaviour [20], [23], [24].

D. NATS with JetStream persistence

The NATS ecosystem deserves separate consideration because it targets lightweight, location-transparent distributed communication. Official NATS documentation presents core NATS as a connective technology for modern distributed systems with publish/subscribe and request/reply communication, while JetStream adds persistence, replay, replication, and consumer state on top of the base message bus [25]–[27]. JetStream explicitly supports replay from the beginning of a stream, from a specific sequence number, or from a time-based position, and it offers both push-based and pull-based consumers for different workload profiles [27].

For the thesis, NATS is interesting because it represents a lightweight alternative to Kafka and RabbitMQ rather than a direct substitute for either. JetStream supports durable streams, replay, replication, and scalable pull consumers, while core NATS preserves a simple request/reply and pub/sub model that is attractive for microservice systems [26], [27]. A key complication is that the literature still often discusses the older NATS Streaming layer rather than JetStream, which means published results do not always map perfectly to the current product. That mismatch should be made explicit in the thesis: the experiment should evaluate modern JetStream, but the literature review must interpret older NATS Streaming results with caution [22], [23]. NATS with JetStream is most promising when lightweight deployment, elastic scaling, and a smaller operational surface are valued more highly than Kafka's larger ecosystem and benchmarking history.

E. Benchmark Design

1) Methodological positioning: The benchmark design is not presented as a standard protocol copied directly from one prior study, nor as a completely novel framework. Instead, it adapts recurring benchmarking principles from streaming-system evaluation, distributed stream processing studies, and comparative IPC experiments to the specific constraints of DYNAMOS. In particular, the controlled multi-stage pipeline follows the style of prior streaming-pipeline evaluations such as DataXc [7]; the insistence on identical business logic across mechanisms follows comparative benchmarking practice in distributed systems [28], [29]; and the emphasis on throughput, latency, concurrency, resource pressure, and recovery reflects recurring evaluation dimensions in the literature [16], [22]. The resulting design is therefore best understood as a literature-guided, thesis-specific operationalization of established evaluation concerns.

2) Evaluation scope: The purpose of this benchmark is not to identify a universally best communication mechanism in the abstract. Instead, it narrows the design space for DYNAMOS by asking which communication model best fits which workload condition and which operational constraint. The central question is therefore: *which streaming mechanism best supports large, ordered, and concurrent inter-service data transfer in a microservice pipeline under varying workload intensity, resource pressure, and recovery requirements?*

This framing matters for interpretation. The chapter body focuses on the argument and on the most informative comparisons, while exhaustive preset definitions, full run matrices, and supplementary plots are better treated as appendix material once the multi-technology comparison is complete.

3) Common testbed and fairness constraints: To ensure that observed differences can be attributed to the communication mechanism rather than to application logic, all candidate mechanisms should be evaluated with the same payload schema, the same transformation logic, and the same deployment environment. The common testbed is a Go-based microservice chain deployed on Kubernetes with three services: *Producer* $\rightarrow$ *Transformer* $\rightarrow$ *Sink*. The services are deployed in a dedicated namespace using Kustomize, with base manifests defining the common topology and overlay patches controlling the resource budget per experiment tier (CPU and memory requests and limits). Resource consumption is sampled at regular intervals via the Kubernetes metrics-server using `kubectl top`. This topology is small enough to reason about directly, but still rich enough to expose backpressure, buffering, and multi-hop propagation effects [7], [28].

The mechanism-specific mappings should remain as close as possible to one another. For gRPC, the default implementation should use client-streaming, with bidirectional streaming reserved for an extended scenario. For Kafka, messages should be keyed by flow identifier so that per-flow ordering is preserved within a partition. For RabbitMQ, the preferred setup is a stream rather than a classic queue, with super streams used only if scale-out beyond a single stream is necessary. For NATS, JetStream should be used with explicit acknowledgements and pull consumers. Holding business logic and payload shape constant is essential, because otherwise performance differences could be caused by application behaviour instead of the communication layer itself [28], [29].

The service logic is intentionally simple. The producer emits synthetic records or chunked data, the transformer applies a deterministic lightweight operation such as validation, filtering, or hashing, and the sink records arrival times, latency summaries, and correctness counters. This keeps the benchmark focused on communication behaviour rather than on domain-specific business processing.

4) Evaluation dimensions: The literature supports evaluating streaming mechanisms along a small set of recurring dimensions rather than through a single headline metric. In this benchmark, five dimensions are treated as primary: transfer efficiency, meaning how much data the pipeline can move per second under comparable conditions; latency and pacing stability, meaning not only how fast messages arrive but how regularly they arrive under sustained flow [7]; overload behaviour, meaning whether pressure is absorbed through queue growth, throttling, or admission failure; recovery and correctness, meaning how reconnects, retries, duplicates, and ordering behave when a running pipeline is interrupted; and re-

source and operational cost, meaning CPU, memory, network traffic, and the practical complexity of running the mechanism in Kubernetes.

This dimensional view is important because it prevents the chapter from collapsing into a single throughput comparison. A mechanism may lead on bulk transport while still performing poorly under restart or overload, and such trade-offs are exactly what DYNAMOS must reason about.

5) Scenario set and scenario-to-property mapping: The full benchmark design distinguishes three workload families: *bulk transfer*, *continuous steady-rate flow*, and *bursty transfer*. The currently completed gRPC reference study covers the first two directly and supplements them with two targeted stress probes that operationalize the properties most relevant to DYNAMOS: slow-consumer backpressure and forced-restart recovery. A bursty profile remains part of the planned cross-technology matrix, but it has not yet been executed in the current gRPC-only reference stage.

6) Experimental factors: The independent variables cover the stress dimensions most frequently discussed in streaming-system and IPC benchmarking literature [16], [22], [29]: dataset size, to expose when buffering or backlog effects become significant; chunk or message size, to test the trade-off between per-message overhead and coarse-grained transfer latency; number of concurrent flows, to assess how each mechanism handles simultaneous transfers; pipeline depth, fixed here at a three-service chain so that raw transport overhead and multi-hop effects can still be distinguished; consumer speed mismatch, to expose backlog behaviour and flow-control effectiveness; resource budget, through explicit CPU and memory requests and limits for Kubernetes pods, defined as Kustomize overlay patches at three tiers (small: 100 m CPU / 128 MiB, medium: 250 m CPU / 256 MiB, large: 500 m CPU / 512 MiB); durability mode and replication factor, where the mechanism supports such trade-offs; and fault scenario, at minimum service restart during an ongoing transfer.

Not every factor needs to be crossed exhaustively. A pragmatic design is to define a core matrix over mechanism, workload profile, resource budget, and concurrency, and then use targeted follow-up experiments for failure and durability scenarios. This keeps the study manageable while still covering the trade-offs the literature treats as most important [22], [23].

7) Result presentation and appendix boundary: The body of the thesis should present only the compact experiment matrix and a smaller set of focused figures that are necessary for the argument. A master table can summarize each run by mechanism, workload profile, dataset size, message size, concurrency level, resource budget, and durability mode, together with the most important metrics. The main text should then highlight the most informative sweeps, such as throughput versus concurrency, latency percentiles versus message size, and recovery behaviour by failure scenario. Full preset definitions, per-run outputs, and secondary plots should be moved to an appendix so that the chapter remains analytically clear rather than turning into an undifferentiated lab log.

F. gRPC Baseline Results

1) Metrics definitions: Before presenting results, it is important to define the metrics used throughout the evaluation so that tables, figures, and prose refer to consistent quantities.

- **Throughput (msg/s)** is the number of messages successfully received by the sink per second of wall-clock time, averaged over the entire run.
- **Throughput (MiB/s)** is the data volume received by the sink per second, computed as $(\mathrm{msg/s}) \times \mathrm{message\ size}$.
- **Latency percentiles (p_{50}, p_{95}, p_{99}, max)** describe end-to-end message delivery delay from producer send to sink receipt. The p_{95} latency, for example, means that 95% of messages were delivered within that time or less.
- **Inter-arrival time** is the elapsed time between consecutive message arrivals at the sink. The p_{95} inter-arrival time captures near-worst-case spacing between deliveries.
- **Jitter** is defined as the standard deviation of the inter-arrival times. Lower jitter indicates a more regular, predictable delivery rhythm.
- **Duplicates** is the number of messages received more than once by the sink. In a correct delivery, this count is zero.
- **Ordering violations** is the number of messages received out of their original send order.
- **CPU usage (%)** is the average and peak CPU consumption per role (producer, transformer, sink), sampled at two-second intervals via `kubectl top pods` during the focused Kubernetes reruns and computed as a percentage of a single core.
- **Memory usage (MiB)** is the average and peak resident memory per role, sampled on the same interval.
- **Recovery time (ms)** is the wall-clock time between the last message received before a failure and the first message received after the affected component has restarted.

2) Current comparison scope: The currently completed implementation covers three transport mappings in the same three-stage pipeline (*Producer* $\rightarrow$ *Transformer* $\rightarrow$ *Sink*): *client-streaming gRPC*, *unary gRPC*, and *RabbitMQ streams*. The two gRPC variants remain informative because unary approximates the current direct request-response style used in DYNAMOS, whereas client-streaming represents the most direct streaming extension of that model. RabbitMQ streams now adds a broker-mediated reference point in which the producer publishes into a durable stream and the downstream stages consume through the broker rather than over a single long-lived RPC. Kafka and NATS remain future work.

Two clean workload families were executed for this reference matrix. The first uses synthetic payloads with message sizes from 256 bytes to 16 KiB and concurrency levels from 1 to 16. The second is a smaller realism check that replays rows from the copied DYNAMOS CSV datasets by batching either 25 or 100 rows into one message. For each run, the sink records throughput, end-to-end latency percentiles, and correctness counters. In addition to these clean matrices, three focused stress scenarios were executed to probe the properties identified earlier: a continuous steady-rate jitter scenario, a

Table I
SCENARIO-TO-PROPERTY MAPPING USED IN THE BENCHMARK DESIGN.

Scenario	Primary property	Why it matters for DYNAMOS
Bulk transfer baseline	Protocol overhead, sustained throughput, multi-hop transfer efficiency	Large dataset exchange should remain efficient when data is moved incrementally through a composed service chain.
Continuous steady-rate flow	Latency stability, inter-arrival regularity, pacing precision, CPU cost	Long-lived flows matter when data is produced incrementally and consumers depend on relatively stable delivery rhythm.
Slow-consumer backpressure	Queue build-up, overload propagation, throttling versus delay accumulation	DYNAMOS pipelines may contain stages with unequal service rates, so overload handling must be explicit and interpretable.
Forced-restart recovery	Reconnect behaviour, duplicates, ordering preservation, recovery time	Ephemeral microservices and restarts are normal in Kubernetes-style deployments, so transport behaviour under interruption is a first-class concern.

slow-consumer backpressure scenario, and a recovery scenario with a forced mid-run transformer restart.

The broad baseline matrices are now split across two environments. The original gRPC bulk-transfer and CSV-replay baselines were executed as single runs per configuration on Docker Compose, where each service runs as an unrestricted container on the same host. The RabbitMQ bulk-transfer and CSV-replay baselines were executed later on a single-node Kubernetes cluster, because the broker path and the corresponding Kubernetes runner support were added after the initial gRPC study. The three focused stress scenarios were then rerun three times on that Kubernetes cluster (with Kustomize-managed resource limits) for all currently implemented transports and are reported below as arithmetic means over those repetitions. This split means that the broad gRPC and RabbitMQ baseline tables should not be read as directly comparable absolute performance numbers. Additionally, the Kubernetes RabbitMQ CSV-replay pilot mounts truncated test copies of the DYNAMOS CSV files through a ConfigMap, because the full copied datasets exceed the practical object-size limits of the cluster control plane.

Table II
GRPC BASELINE RESULTS AVERAGED OVER THE COMPLETED RUN MATRICES.

Workload	Transport mode	Avg. throughput (msg/s)	Avg. p_{95} latency (ms)
Synthetic	Client-streaming	73,087.66	42.82
Synthetic	Unary	5,420.28	1.44
CSV replay	Client-streaming	54,814.09	19.24
CSV replay	Unary	4,375.15	1.06

3) Clean baseline matrix: Table IV breaks the synthetic bulk-transfer results down by message size, averaging across

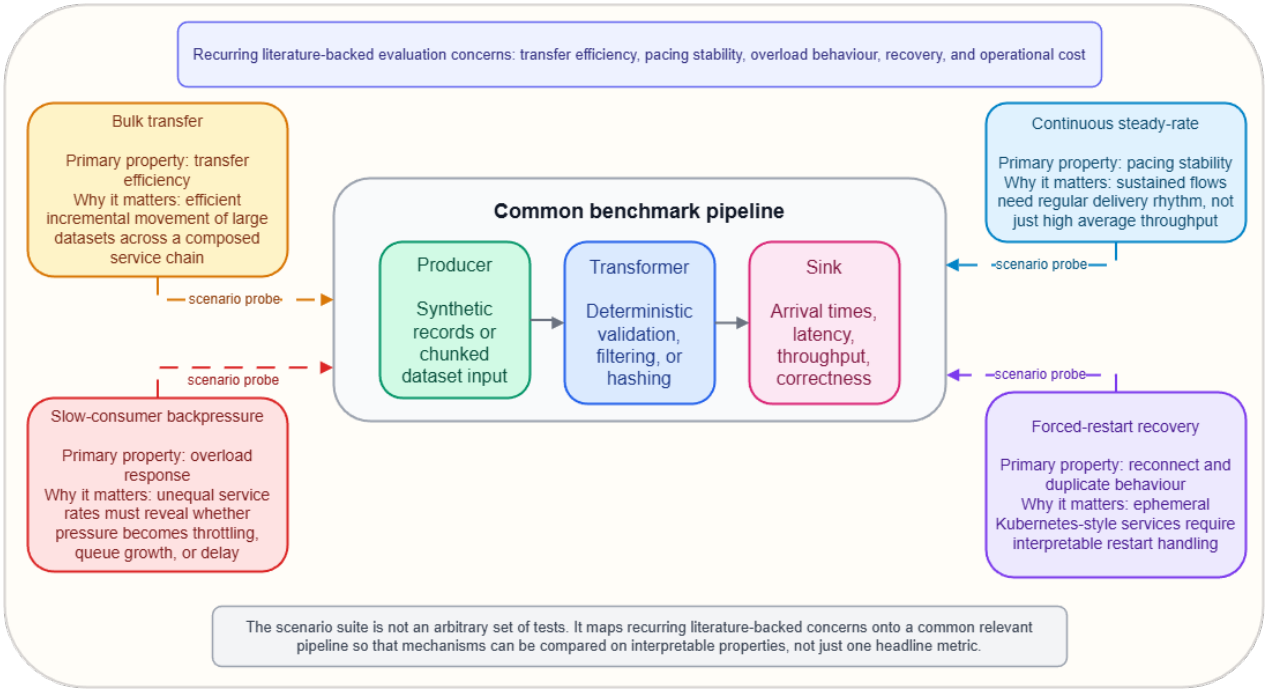

Figure 1. Evaluation map showing the common benchmark pipeline and the four scenario probes used to operationalize the literature-backed evaluation dimensions.

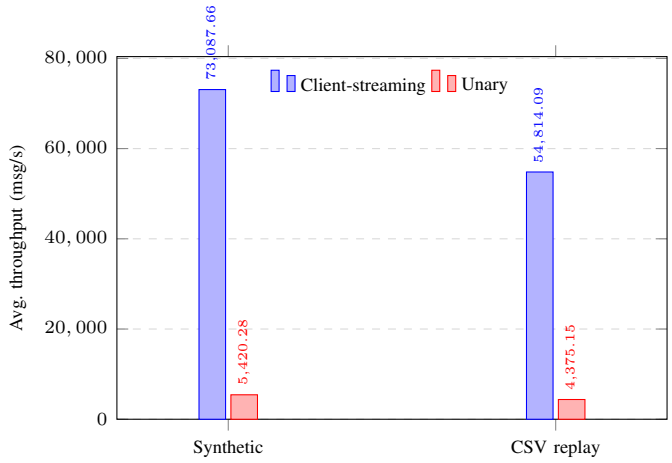

Figure 2. Average throughput for the gRPC baseline, split by workload family and transport mode. Client-streaming sustains substantially higher throughput than unary gRPC in both workload families.

Table III
RABBITMQ STREAMS PILOT RESULTS AVERAGED OVER THE COMPLETED KUBERNETES BASELINE MATRICES.

Workload	Avg. throughput (msg/s)	Avg. p_{95} latency (ms)
Synthetic	312.43	20,137.58
CSV replay	361.41	8,632.07

concurrency levels 1–16 for each size. This disaggregation reveals that the streaming advantage is strongly size-dependent: client-streaming achieves roughly 21× the throughput of unary at 256 B but only 4.8× at 16 KiB. The grand averages in Table II (approximately 13.5× for synthetic) therefore overstate the advantage for larger messages and understate it for smaller ones.

Two observations stand out. First, streaming throughput in msg/s drops substantially with message size (from 136,K to 21,K), but data throughput in MiB/s rises (from 33 to 335 MiB/s) because larger messages amortise per-message overhead more effectively. Unary msg/s, by contrast, remains relatively flat across sizes (4.5–6.4 K), indicating that per-call overhead rather than bandwidth capacity is the dominant bottleneck. Second, unary p_{95} latency stays below 1.7 ms at every message size, whereas streaming p_{95} latency varies between 29 and 80 ms depending on size and concurrency. This confirms that unary remains the lower-latency option on a per-message basis. The CSV replay workload (Table II) shows a broadly consistent pattern with a 12.5× throughput ratio.

The RabbitMQ pilot baseline in Table III occupies a very different operating region. Average throughput is only 312 messages per second for the synthetic clean matrix and 361 messages per second for the CSV replay matrix, while average p_{95} latency rises into the multi-second range. Across the synthetic size sweep, throughput in msg/s remains nearly flat at roughly 309–320 messages per second, but the data rate still increases from about 0.08 MiB/s at 256 B to about 4.93 MiB/s at 16 KiB as larger payloads amortise the broker overhead more effectively. The clean RabbitMQ runs did, however, preserve correctness: no duplicate deliveries or ordering violations were observed in the baseline matrices, so the current limitation is performance rather than obvious delivery corruption.

Third, the clean baseline matrices did not reveal correctness problems in the implemented gRPC benchmark: no duplicate deliveries or ordering violations were observed in those completed bulk-transfer runs. This is important because raw throughput alone would not be useful if it came at the cost of lost ordering or inconsistent delivery.

The clean matrices alone, however, are not enough to judge whether a transport is suitable for long-lived, policy-constrained data exchange. DYNAMOS-style workflows also care about pacing regularity, behaviour under consumer mismatch, and failure recovery. Three focused stress scenarios were therefore added after the baseline.

4) Focused stress scenarios:

a) Continuous steady-rate scenario.: The first focused experiment was a continuous steady-rate scenario. It used 1 KiB synthetic messages, a fixed target rate of 1,000 messages per second, and concurrency levels of 4 and 8, without artificial sink delay or injected failures. The goal was to measure not just throughput and latency, but also inter-arrival regularity under a long-running steady stream.

The repeated continuous results show a clear split between delivery regularity and per-message responsiveness. Throughput was effectively identical in the four gRPC cases at roughly 919 messages per second, so the difference there lies in how the flow is shaped. Client-streaming maintained the

Table IV
SYNTHETIC BULK-TRANSFER THROUGHPUT BY MESSAGE SIZE, AVERAGED ACROSS CONCURRENCY LEVELS 1–16 (DOCKER COMPOSE, SINGLE RUNS).

Msg size	Stream msg/s	Stream MiB/s	Unary msg/s	Unary MiB/s	Ratio
256 B	135,827	33.2	6,449	1.6	21.1×
1 KiB	101,142	98.8	5,071	5.0	19.9×
4 KiB	33,920	132.5	5,648	22.1	6.0×
16 KiB	21,462	335.3	4,513	70.5	4.8×

Table V
FOCUSED CONTINUOUS-STREAM SCENARIO AVERAGED OVER THREE KUBERNETES REPETITIONS.

Transport mode	Concur.	Throughput	Avg. p_{95}	p_{95} inter-arrival	Jitter
Client-streaming	4	918.81	12.64	1.66	0.72
Client-streaming	8	918.98	13.07	1.71	0.81
Unary	4	918.88	2.26	2.10	2.13
Unary	8	918.71	5.49	2.09	1.58
RabbitMQ streams	4	395.41	14,643.54	3.61	9.44
RabbitMQ streams	8	357.22	17,193.05	5.55	10.12

Table VI
FOCUSED BACKPRESSURE SCENARIO WITH AN INTENTIONALLY SLOW SINK, AVERAGED OVER THREE KUBERNETES REPETITIONS.

Transport mode	Throughput	Median lat.	Avg. p_{95}	p_{95} inter-arrival	Jitter
Client-streaming	2,324.34	1,588.75	4,352.82	2.10	0.64
Unary	1,771.80	0.45	0.86	3.24	1.10
RabbitMQ streams	366.80	11,924.53	16,161.30	6.10	9.50

Table VII
FOCUSED RECOVERY SCENARIO WITH A FORCED TRANSFORMER RESTART, AVERAGED OVER THREE KUBERNETES REPETITIONS.

Transport mode	Throughput	Avg. p_{95}	Max lat.	Duplicates	Avg. recovery
Client-streaming	1,278.49	28.28	4,851.44	9,783	335.12
Unary	589.39	3.52	25,549.77	0	22,822.09
RabbitMQ streams	340.87	20,365.46	34,762.02	590	0.00

tighter inter-arrival pattern in both concurrency settings: its jitter stayed below 0.81 ms and its p_{95} inter-arrival delay remained near 1.7 ms. Unary delivered much lower end-to-end latency, with average p_{95} latency between 2.26 and 5.49 ms instead of roughly 13 ms, but its inter-arrival spread was wider and less stable. The CPU samples point in the same direction. Where measurements were consistently available, unary required more transformer and sink CPU than client-streaming, indicating that the lower latency was achieved by spending more processing effort to keep individual request-response exchanges moving. RabbitMQ streams sits in a different regime altogether: throughput drops to 395 and 357 messages per second, while p_{95} latency rises to roughly 14.6 and 17.2 seconds. Its p_{95} inter-arrival delay remains in the low single-digit milliseconds, but that regularity is dominated by accumulated queueing delay in the brokered path rather than by fast end-to-end delivery.

b) Slow-consumer backpressure scenario.: The second focused experiment was a backpressure scenario. It used the same 1 KiB payload size with concurrency 8, but the single sink instance was slowed down by adding a 2 ms processing delay per message while the producer still targeted 4,000 messages per second. Because the sink can process at most $\lfloor 1000/2 \rfloor = 500$ messages per second per connection, the aggregate theoretical sink capacity is $500 \times 8 = 4{,}000$ messages per second, which means the scenario operates at the boundary of the sink's processing budget. Any transport overhead or scheduling jitter pushes the effective throughput below the requested rate, causing messages to accumulate either in the gRPC transport buffers of the transformer or in the RabbitMQ broker path. This scenario was designed to expose whether the transport absorbs overload through explicit throttling or through queue growth and accumulated delay.

This result is important because it shows that higher throughput is not automatically the better outcome. Under backpressure, client-streaming kept substantially more of the nominal input rate, but it did so by allowing the pipeline to accumulate a very large queue. The median end-to-end latency increased to about 1.59 seconds and the average p_{95} latency rose to 4.35 seconds. Unary behaved almost oppositely: it gave up throughput, but kept latency near the sub-millisecond range for successfully delivered requests. The interpretation is that client-streaming preserves bulk movement under pressure by buffering aggressively, whereas unary currently protects responsiveness by throttling harder and accepting a lower realized send rate. RabbitMQ streams pushed this queue-absorbing behaviour further: it sustained only 367 messages per second, yet still accumulated about 11.9 seconds of median end-to-end delay and a p_{95} latency above 16 seconds. In the current brokered prototype, overload is therefore translated primarily into persistent backlog and long delivery delay rather than into improved realized throughput.

c) Forced-restart recovery scenario.: The third focused experiment was a recovery scenario. This run used 4 KiB messages, concurrency 8, a continuous target rate of 2,000 messages per second, and a forced *Transformer* restart after 3 seconds. The producer was allowed to retry up to 120 times with a 250 ms backoff. The purpose of this scenario was not to replace the main matrix, but to observe how the currently implemented transport mappings behave when the pipeline is interrupted during an active stream.

The repeated recovery results change the interpretation in an important way. Client-streaming, not unary, preserved the flow much more effectively after the restart. It recovered in about 335 ms on average, sustained 1,278 messages per second across the run, and limited worst-case latency to about 4.85 seconds. Unary still kept a much lower p_{95} latency for successfully delivered requests, and it completed the run without duplicates, but it did so while the overall stream stalled for much longer: the average recovery interval exceeded 22.8 seconds, throughput dropped to 589 messages per second, and the worst-case latency exceeded 25.5 seconds. RabbitMQ streams changes the semantics rather than mirroring either gRPC row directly. Because the producer publishes into a broker, the forced *Transformer* restart is not observed as a producer-side reconnect event, so the average recovery interval remains zero in the table. The sink-side outcome is nonetheless not free: throughput stays near 341 messages per second, p_{95} latency exceeds 20.3 seconds, and about 590 duplicate messages are observed on average during replay after the restart.

The trade-off is that the current client-streaming recovery strategy replays a worker's sequence set after reconnect, which preserves completeness but introduces at-least-once behaviour under failure. This explains the average 9,783 duplicate messages recorded in the streaming recovery runs. It is worth noting that the streaming recovery time showed high variance across repetitions, ranging from sub-millisecond (essentially instant reconnect in two of three runs) to roughly one second, so the 335 ms mean should be interpreted with caution. Unary recovery, by contrast, was highly consistent across repetitions (22.6–23.1 s). Unary avoids duplicate replay, but in this Kubernetes setting it pays for the cleaner sink-side outcome with a

much longer and more predictable interruption window.

The resource metrics reinforce this distinction, but in the opposite direction from the earlier single-run result. In the repeated recovery preset, client-streaming consumed more sampled CPU across all three roles: producer CPU averaged 29.9% versus 5.46% for unary, transformer CPU averaged 10.53% versus 2.23%, and sink CPU averaged 4.52% versus 2.37%. Memory stayed low for both modes, remaining in the single-digit MiB range per role. This suggests that client-streaming keeps more of the pipeline active during and after the restart, whereas unary falls into a lower-throughput, lower-CPU retry regime.

5) Discussion: The current results support a more precise conclusion than a simple "streaming beats unary" claim. In the clean bulk-transfer baseline, client-streaming is the stronger choice for high-volume inter-service data movement, with the throughput advantage varying from roughly $5\times$ for 16 KiB messages to $21\times$ for 256 B messages. Unary remains the lower-latency option for small request-response exchanges, with sub-2 ms p_{95} latency at every message size tested. In the continuous scenario, client-streaming also provides the steadier delivery rhythm, whereas unary provides lower latency at higher CPU cost and with less precise pacing. In the backpressure scenario, client-streaming preserves throughput by absorbing overload as queueing delay, while unary sacrifices throughput to protect latency. Under restart stress, the repeated Kubernetes reruns show that client-streaming now offers the stronger continuity story in this prototype, but only by accepting at-least-once replay and the need for downstream deduplication. Unary remains the cleaner duplicate-free option, yet its retry behaviour currently stretches recovery time and depresses throughput too severely to treat it as the better resilience mechanism overall.

RabbitMQ streams adds a third point in this trade-off space. It preserved correctness in the clean and CSV-replay baselines and reduced producer-visible recovery events to zero under the forced transformer restart, but it did so with much lower sustained throughput and very high end-to-end latency in every evaluated scenario. The broker itself also carries a visible operational cost in the focused Kubernetes runs, consuming roughly 31–34% average CPU and 323–487 MiB of memory in the continuous and recovery presets. In its current single-broker form, RabbitMQ therefore behaves more like a decoupling and backlog-tolerant buffering layer than a performance-oriented replacement for direct gRPC streaming.

These findings already make a useful thesis contribution because they identify concrete transport trade-offs in bulk transfer, pacing regularity, overload response, and failure recovery across both direct RPC and broker-mediated transport styles. That evidence clarifies which properties later candidates must improve and which compromises they may introduce in return.

Kafka and NATS still need to be benchmarked under the same workload definitions, but the current gRPC baseline and the first RabbitMQ streams comparison have already narrowed the evaluation space. The remainder of the thesis can therefore focus on whether other broker-based mechanisms offer meaningful gains in replay, decoupling, or backlog tolerance without further undermining the latency, recovery, and operational properties exposed here.

G. Threats to Validity

Several threats to validity should be acknowledged when interpreting the current results.

a) Internal validity.: The broad synthetic and CSV-replay matrices in the original gRPC baseline were executed as single runs per configuration on Docker Compose, so those portions of the study may be affected by transient system-level effects such as CPU scheduling variance, garbage collection pauses, or network contention from co-located workloads. To reduce that risk for the most thesis-critical claims, the three focused stress scenarios were rerun three times on Kubernetes and the reported tables use arithmetic means across those repetitions. RabbitMQ was introduced after that initial baseline and therefore its broad matrices were also executed on Kubernetes rather than Docker Compose. Even so, the repeat count remains small and no confidence intervals are reported yet. The two-environment split means that absolute numbers are not directly comparable across all broad-baseline sections, though the relative comparisons between transport modes remain valid within each environment. Additionally, short run durations (typically under one minute for bulk-transfer configurations) may not fully capture warm-up effects or long-term stability behaviour that would emerge in sustained production workloads. The Kubernetes RabbitMQ CSV-replay pilot also uses truncated test snapshots of the copied DYNAMOS CSVs because the full files exceeded ConfigMap size limits; the replay source cycles over the loaded rows, so the workload remains valid as a realism check, but the payload mix is narrower than in the full Docker-based CSV baseline.

b) External validity.: The benchmark uses a synthetic three-stage pipeline with lightweight deterministic logic, which does not replicate the full complexity of a DYNAMOS deployment. Real DYNAMOS chains may involve additional processing stages, policy-enforcement sidecars, and heterogeneous service implementations. Furthermore, the tests are conducted in a single-node Kubernetes cluster, whereas production DYNAMOS deployments span multiple nodes with network partitioning and scheduling constraints that may affect latency and throughput characteristics differently. The Go-based benchmark services share the same runtime and gRPC library version, so results may not generalize to polyglot microservice chains.

c) Construct validity.: Jitter is operationalized as the standard deviation of inter-arrival times, which captures variability in delivery rhythm but does not distinguish between periodic jitter and isolated outliers. Alternative metrics such as mean absolute deviation or percentile-based spread could complement the current definition. Similarly, the recovery time metric measures wall-clock gap between the last pre-failure and first post-failure message, but does not account for the

quality of the recovery (e.g., whether the resumed stream carries duplicates or ordering violations).

IV. Planned: Cross-Technology Benchmark Comparison

The benchmark design, the gRPC baseline, and the first RabbitMQ streams comparison presented in section III address the first part of *RQ1*. The planned next step extends that comparison by evaluating Apache Kafka and NATS JetStream under the same workload definitions, fairness constraints, and evaluation dimensions, then revisits the current RabbitMQ findings against that broader broker set. These results will allow a structured recommendation of which mechanism best fits which workload condition, completing the answer to RQ1.

V. Planned: Architectural Integration Design

This section will address *RQ2*: how streaming communication can be integrated into DYNAMOS' microservice architecture while preserving dynamic service composition, policy-driven orchestration, and isolation guarantees. The work will examine how the selected streaming model aligns with the data-sharing archetypes implemented by DYNAMOS, specify how streaming channels are created and torn down within dynamically composed execution chains, define how sidecar responsibilities are divided for streaming versus non-streaming traffic, and address how long-lived connections interact with policy enforcement boundaries. The output will be a documented integration design accompanied by a prototype implementation that demonstrates feasibility.

VI. Planned: System-Level Evaluation

This section will address *RQ3*: what impact streaming communication has on scalability, performance, and robustness compared to the current request-response approach under realistic workloads in DYNAMOS. The benchmark logic and evaluation dimensions established in the current study will be reused, but applied within the full DYNAMOS context rather than in an isolated benchmark pipeline. The evaluation will assess whether the trade-offs identified in the gRPC baseline and cross-technology comparison carry over to the integrated system, or whether integration-specific effects such as policy enforcement overhead, sidecar latency, or orchestration coordination dominate.

VII. Conclusion

This thesis establishes the first empirical basis for evaluating streaming communication in DYNAMOS. The literature review narrows the relevant design space to direct gRPC streaming and broker-based alternatives such as Kafka, RabbitMQ streams, and NATS JetStream. The current benchmark shows that client-streaming gRPC is markedly stronger for sustained bulk transfer, pacing stability, and restart continuity, while unary gRPC remains preferable when low per-message latency and duplicate-free delivery dominate. RabbitMQ streams now provides a first broker-based reference point: it offers decoupling and suppresses producer-visible recovery events, but in the current prototype it does so with much lower throughput, much higher end-to-end latency, and a non-trivial broker resource cost. Rather than yielding a single winner, the present results identify a concrete trade-off space around throughput, regularity, overload handling, recovery, and buffering semantics.

That evidence provides a defensible baseline for the remainder of the thesis. The planned next phases extend the broker comparison beyond RabbitMQ, examine how the most promising options align with DYNAMOS' archetypes and policy model, and translate those findings into an integration design and system-level validation.

Appendix

This appendix documents the complete preset definitions used in the benchmark. Each preset specifies the workload family, payload configuration, concurrency level, target rate (where applicable), failure injection parameters, and resource overlay tier. The five presets are: *synthetic-clean* (bulk transfer baseline), *csv-replay-check* (realism validation), *synthetic-continuous* (steady-rate pacing), *synthetic-backpressure* (slow-consumer overload), and *synthetic-recovery* (forced-restart resilience).

Table VIII
Benchmark presets used in the current transport study.

Preset	Key parameters	Purpose
`synthetic-clean`	Synthetic payloads from 256 B to 16 KiB, concurrency 1–16, no injected failure	Baseline bulk-transfer throughput, latency, and correctness matrix
`csv-replay-check`	Copied DYNAMOS CSV rows batched as 25 or 100 rows per message	Realism check against non-synthetic payload structure
`synthetic-continuous`	1 KiB payloads, 1,000 msg/s target, concurrency 4 and 8, three repetitions	Steady-rate pacing and jitter comparison
`synthetic-backpressure`	1 KiB payloads, 4,000 msg/s target, concurrency 8, 2 ms sink delay, three repetitions	Slow-consumer overload and queueing behaviour
`synthetic-recovery`	4 KiB payloads, 2,000 msg/s target, concurrency 8, transformer restart after 3 s, 120 retries at 250 ms backoff, three repetitions	Restart recovery, replay, and continuity analysis

This appendix summarizes the run scopes used in the current thesis draft. The body of the thesis presents the most decision-relevant subsets, while the benchmark result directory stores the corresponding per-run JSON, NDJSON, and CSV exports generated by the execution scripts. The focused scenarios are repeated; the broad matrices are currently single-run baselines.

This appendix summarizes the resource metrics for the repeated Kubernetes reruns where CPU and memory data are most informative. For RabbitMQ streams, the broker is reported as a separate role because its buffering layer is a core part of the transport cost. The backpressure preset produced too few stable `kubectl top` samples to support a meaningful per-role comparison across all transports, so the appendix focuses on the continuous and recovery scenarios.

References

[1] J. Stutterheim, "Dynamos: Dynamically adaptive microservice-based os: A middleware for data exchange systems," Master's thesis, University of Amsterdam, 2023.

Table IX
EXECUTION SCOPE OF THE CURRENT BENCHMARK CORPUS.

Scope	Transports	Variable dimensions	Repetitions	Role in thesis
Synthetic clean matrix	Client-streaming, unary, rabbitmq-streams	Payload size and concurrency sweep	1	Main bulk-transfer baseline in Section III
CSV replay check	Client-streaming, unary, rabbitmq-streams	Batch size and concurrency subset	1	Realism validation against copied DYNAMOS datasets
Focused continuous	Client-streaming, unary, rabbitmq-streams	Concurrency 4 and 8 at fixed 1 KiB and 1,000 msg/s	3	Pacing and jitter evidence
Focused backpressure	Client-streaming, unary, rabbitmq-streams	Fixed 1 KiB, concurrency 8, 4,000 msg/s, 2 ms sink delay	3	Overload and queueing evidence
Focused recovery	Client-streaming, unary, rabbitmq-streams	Fixed 4 KiB, concurrency 8, forced transformer restart	3	Recovery and replay evidence

Table X
CONTINUOUS SCENARIO CPU USAGE BY ROLE (AVG./PEAK %).

Mode	Concur.	Producer	Transformer	Sink	Broker
Client-streaming	4	26.10 / 26.10	10.54 / 25.63	6.11 / 14.77	–
Client-streaming	8	–	9.75 / 25.53	5.69 / 14.80	–
Unary	4	34.40 / 34.40	14.63 / 39.27	8.18 / 21.93	–
Unary	8	37.83 / 37.83	16.50 / 42.10	9.05 / 23.10	–
RabbitMQ streams	4	7.81 / 9.43	7.76 / 14.12	4.55 / 8.55	30.72 / 50.00
RabbitMQ streams	8	7.28 / 8.90	7.92 / 13.23	4.61 / 7.93	32.90 / 50.03

Table XI
RECOVERY SCENARIO CPU USAGE BY ROLE (AVG./PEAK %).

Mode	Producer	Transformer	Sink	Broker
Client-streaming	29.90 / 29.90	10.53 / 26.17	4.52 / 17.33	–
Unary	5.46 / 5.60	2.23 / 9.27	2.37 / 5.73	–
RabbitMQ streams	7.82 / 9.68	10.69 / 16.68	4.67 / 9.78	33.24 / 50.00

Table XII
RECOVERY SCENARIO MEMORY USAGE BY ROLE (AVG./PEAK MIB).

Mode	Producer	Transformer	Sink	Broker
Client-streaming	8.00 / 8.00	3.60 / 6.00	3.04 / 7.00	–
Unary	7.00 / 7.00	3.14 / 7.00	4.90 / 7.00	–
RabbitMQ streams	7.31 / 7.50	7.45 / 8.00	5.97 / 8.00	486.71 / 511.00

[2] J. Stutterheim, A. Mifsud, and A. Oprescu, "Dynamos: Dynamic microservice composition for data-exchange systems, lessons learned," in *Proceedings of the IEEE International Conference on Software Architecture (ICSA)*, 2023.

[3] C. Starck and J. Ghofrani, "Improving load balancing of long-lived streaming rpcs for grpc-enabled inter-service communication," in *Proceedings of the 15th Central European Workshop on Services and their Composition (ZEUS 2023)*, ser. CEUR Workshop Proceedings, S. Böhm and D. Lübke, Eds., vol. 3386, Hannover, Germany, 2023, pp. 45–51. [Online]. Available: https://ceur-ws.org/Vol-3386/

[4] A. Sodankoor, A. Shenoy, B. M. Moger, M. Gowda, P. K. Auradkar, and S. Kalambur, "Benchmarking grpc protocol on various virtualization technologies," in *ICT4SD 2025*, ser. Lecture Notes in Networks and Systems. Springer Nature Switzerland AG, 2026, vol. 1654, pp. 316–329.

[5] A. G. Ibrahim, R. P. Lopes, J. Rufino, and P. Leitão, "On the impact of message brokers implementations in the choreography of microservices," in *Open Lecture Series 2A (OL2A 2025)*, ser. Communications in Computer and Information Science (CCIS). Springer Nature Switzerland AG, 2026.

[6] J. Nuikka, "Comparison of cloud native messaging technologies," Master's thesis, Tampere University, Faculty of Information Technology and Communication Sciences, May 2021. [Online]. Available: https://urn.fi/URN:NBN:fi:tuni-202104223314

[7] G. Coviello, K. Rao, C. G. D. Vita, G. Mellone, and S. Chakradhar, "Dataxc: Flexible and efficient communication in microservices-based stream analytics pipelines," in *2022 IEEE Intl Conf on Dependable, Autonomic and Secure Computing (DASC) / PiCom / CBDCom / CyberSciTech*, 2022.

[8] J. Figueira and C. Coutinho, "Developing self-adaptive microservices," *Procedia Computer Science*, vol. 232, pp. 264–273, 2024.

[9] S. Neumaier, G. Havur, and T. Pellegrini, "Towards an architecture for policy-aware decentral dataset exchange," in *Proceedings of the Fifteenth International Conference on Advances in Semantic Processing (SEMAPRO 2021)*. Barcelona, Spain: IARIA, Oct. 2021. [Online]. Available: https://www.researchgate.net/publication/358816711_Towards_an_Architecture_for_Policy-Aware_Decentral_Dataset_Exchange

[10] G. Thiyagarajan, V. Bist, and P. Nayak, "Strengthening grpc security in microservices: A proxy-based approach for mtls, jwt, and rbac enforcement," *International Journal of Computer Applications*, vol. 187, no. 28, Aug. 2025.

[11] J. A. Rasheedh and S. Saradha, "Reactive microservices architecture using a framework of fault tolerance mechanisms," in *Proceedings of the Second International Conference on Electronics and Sustainable Communication Systems (ICESC)*. IEEE, 2021.

[12] B. Guntupalli and S. Vamshi Ch, "Designing microservices that handle high-volume data loads," *International Journal of AI, Big Data, Computational and Management Studies*, vol. 4, no. 4, pp. 76–87, 2023.

[13] Y. Gan and C. Delimitrou, "The architectural implications of cloud microservices," *IEEE Computer Architecture Letters*, vol. 17, no. 2, pp. 155–158, 2018.

[14] Ritu, S. Arora, A. Bhardwaj, A. Kukkar, and S. Kaur, "A comparative analysis of communication efficiency: Rest vs. grpc in microservice-based ecosystems," in *Proceedings of the 2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)*. IEEE, 2024.

[15] V. Lähtevänoja, "Communication methods and protocols between microservices on a public cloud platform," Master's thesis, Aalto University, School of Science, Espoo, Finland, May 2021. [Online]. Available: https://urn.fi/URN:NBN:fi:aalto-202106207501

[16] S. Ris, J. Araujo, and D. Beserra, "A systemic mapping of methods and tools for performance analysis of data streaming with containerized microservices architecture," in *2023 18th Iberian Conference on Information Systems and Technologies (CISTI)*, 2023, pp. 1–6.

[17] gRPC Authors. (2024) Core concepts, architecture and lifecycle. Last modified 2024-11-12. [Online]. Available: https://grpc.io/docs/what-is-grpc/core-concepts/

[18] M. Roohitavaf, K. Ren, G. Zhang, and S. Ben-Romdhane, "Logplayer: Fault-tolerant exactly-once delivery using grpc asynchronous streaming," 2019, arXiv preprint.

[19] Apache Software Foundation. (2026) Apache kafka. [Online]. Available: https://kafka.apache.org/

[20] P. Dobbelaere and K. Sheykh Esmaili, "Kafka versus rabbitmq," 2017, arXiv preprint.

[21] M. Mohammad, "Analysis of design patterns and benchmark practices in apache kafka event-streaming systems," 2025, arXiv preprint.

[22] B. Çiftçi and B. Çiloğlugil, "A review of comparative studies on performance evaluation of communication mechanisms for microservices," in *Computational Science and Its Applications – ICCSA 2025*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland AG, 2025, vol. 15650, pp. 98–113.

[23] T. Sharvari and K. Sowmya Nag, "A study on modern messaging systems: Kafka, rabbitmq and nats streaming," 2019, arXiv preprint.

[24] RabbitMQ Team. (2026) Streams and superstreams (partitioned streams). [Online]. Available: https://www.rabbitmq.com/docs/streams

[25] The NATS Authors. (2026) Overview. [Online]. Available: https://docs.nats.io/nats-concepts/overview

[26] ——. (2026) Request-reply. [Online]. Available: https://docs.nats.io/nats-concepts/core-nats/reqreply

[27] ——. (2026) Jetstream. [Online]. Available: https://docs.nats.io/nats-concepts/jetstream

[28] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proceedings of the IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1519–1530.

[29] S. Henning and W. Hasselbring, "How to measure scalability of distributed stream processing engines?" in *Companion of the ACM/SPEC International Conference on Performance Engineering (ICPE '21)*. ACM, 2021, pp. 85–88.